



American International University-Bangladesh (AIUB)

Department of Computer Science
Faculty of Science & Technology (FST)

Introduction to Data Science [K]

Mid-Term Project Report

PROJECT TITLE:

Data Cleaning, Preprocessing, and Exploratory Data Analysis of the Stroke Prediction Dataset

Semester:

Summer 2025-26

Supervised by:

Kamrun Naher Koli

<i>Group: 04</i>		<i>Section: K</i>	<i>Contribution</i>
SL	Student Name	Student ID	
1	MD. MURAD HASAN	23-55559-3	25%
2	MAHBUBA NASRIN	23-54976-3	25%
3	ATIA CHOWDHURY	23-52252-2	25%
4	NAJIAT ISLAM RISHAD	23-55218-3	25%

Table of Contents

About the Dataset.....	1
1. Introduction.....	1
2. Dataset Description.....	2
3. Data Issues Identified	2
4. Data Cleaning & Preparation	3
I. Import the Dataset into R	3
II. Dataset Understanding	4
III. Missing Values	5
IV. Barplot of the Missing Values	6
V. Inconsistent Data.....	7
VI. Standardize Inconsistent Data	7
VII. Invalid Data (Gender)	8
VIII. Outlier Detection (Age)	8
IX. Handle Age Outliers	9
X. Hypertension & Heart Disease: Fix Invalid Data	10
XI. Gender & Ever_Married (Fix the Null Values).....	11
XII. BMI (Fix Null Values).....	11
XIII. Final Missing Value Check.....	12
XIV. Type Conversion.....	12
XV. Normalization	13
XVI. Data Filtering.....	13
5. Exploratory Data Analysis	14
I. Descriptive Statistics	14
II. Compare Average Glucose Level by Stroke Group.....	14
III. Compare BMI Spread Across Smoking Status.....	15
IV. Correlation Analysis.....	16
6. Conclusion	17

About the Dataset

This project uses the Stroke Prediction Dataset, originally published by fedesoriano on Kaggle (<https://www.kaggle.com/datasets/fedesoriano/stroke-prediction-dataset>), specifically the file healthcare-dataset-stroke-data.csv. It contains 5,110 patient records across 12 columns: a patient ID, ten health and demographic predictor variables (gender, age, hypertension, heart disease, marital status, work type, residence type, average glucose level, BMI, and smoking status), and one binary target variable, stroke (1 = patient has had a stroke, 0 = has not). For this project, a copy of the original dataset was deliberately modified to introduce missing values, invalid category codes, inconsistent text formatting, and outliers, so that the full data cleaning and preparation pipeline could be demonstrated on realistic problems.

1. Introduction

Stroke remains one of the most serious global health concerns. Global Burden of Disease 2021 estimates show that among non-communicable disorders, stroke is the second leading cause of death, responsible for about 7 million deaths, and the third leading cause of death and disability combined when measured in disability-adjusted life years. Because stroke risk is closely tied to measurable factors such as age, blood glucose level, hypertension, heart disease, and lifestyle habits, healthcare datasets like this one are widely used to study which patient characteristics are most associated with stroke occurrence.

In practice, however, real-world healthcare data is rarely collected without errors — patient records can have missing fields, inconsistent category labels (e.g., a Yes/No field recorded as "yes", "Y", or "1"), invalid entries outside expected ranges, and occasional data-entry mistakes that produce impossible values. Before any meaningful analysis can be trusted, this kind of noise has to be identified and corrected.

This project works with the Stroke Prediction Dataset described above. To simulate a realistic data quality problem, several issues were deliberately introduced into a copy of the clean dataset: missing values across five columns (gender, age, hypertension, heart_disease, ever_married), invalid numeric codes in the hypertension and heart_disease fields, inconsistent text formatting in ever_married and gender, and unrealistic outlier values in age.

The remainder of this report first identifies and documents these issues, then applies appropriate data cleaning and preparation techniques to correct them, and finally performs exploratory data analysis to calculate and interpret descriptive statistics for the cleaned dataset. Predictive modeling (e.g., building a classifier for stroke risk) is outside the scope of this phase and is left for a later stage of the project.

2. Dataset Description

The dataset consists of 5,110 patient records and 12 columns, described below:

<u>Column</u>	<u>Description</u>
id	Unique patient identifier
gender	Male, Female, or Other
age	Patient's age in years
hypertension	0 = no hypertension, 1 = has hypertension
heart_disease	0 = no heart disease, 1 = has heart disease
ever_married	Yes or No
work_type	Private, Self-employed, Govt_job, children, or Never_worked
Residence_type	Urban or Rural
avg_glucose_level	Average blood glucose level (mg/dL)
bmi	Body Mass Index
smoking_status	formerly smoked, never smoked, smokes, or Unknown
stroke	Target variable — 1 = patient has had a stroke, 0 = has not

Ten columns are predictor variables and one (stroke) is the target. Three variables (age, avg_glucose_level, bmi) are numeric/continuous; the rest are categorical.

3. Data Issues Identified

To create a realistic data cleaning task, the following issues were intentionally introduced into a copy of the original clean dataset:

- **Missing values** — gender (145 rows), age (100 rows), hypertension (70 rows), heart_disease (70 rows), and ever_married (80 rows) contain missing entries. (bmi also has 201 missing values, but these occur naturally in the original dataset and were not introduced.)
- **Invalid/incorrect entries** — hypertension and heart_disease should only contain 0 or

1, but 100 rows each contain out-of-range codes such as -1, 2, 3, 5, and 99. gender contains 15 rows with an invalid "unknown" label or blank text instead of Male/Female/Other.

- **Inconsistent formatting** — ever_married is recorded inconsistently across 60 rows, with variants such as "yes", "no", "Y", "N", "1", "Married", and blank entries alongside the standard "Yes"/"No".
- **Outliers** — age contains 80 rows with impossible values, including negative ages and values as high as 990.
- **Duplicate rows** — checked and none found; this will still be verified with code in the cleaning section for completeness.

In total, about 766 of the 5,110 rows ($\approx 15\%$) contain at least one introduced issue.

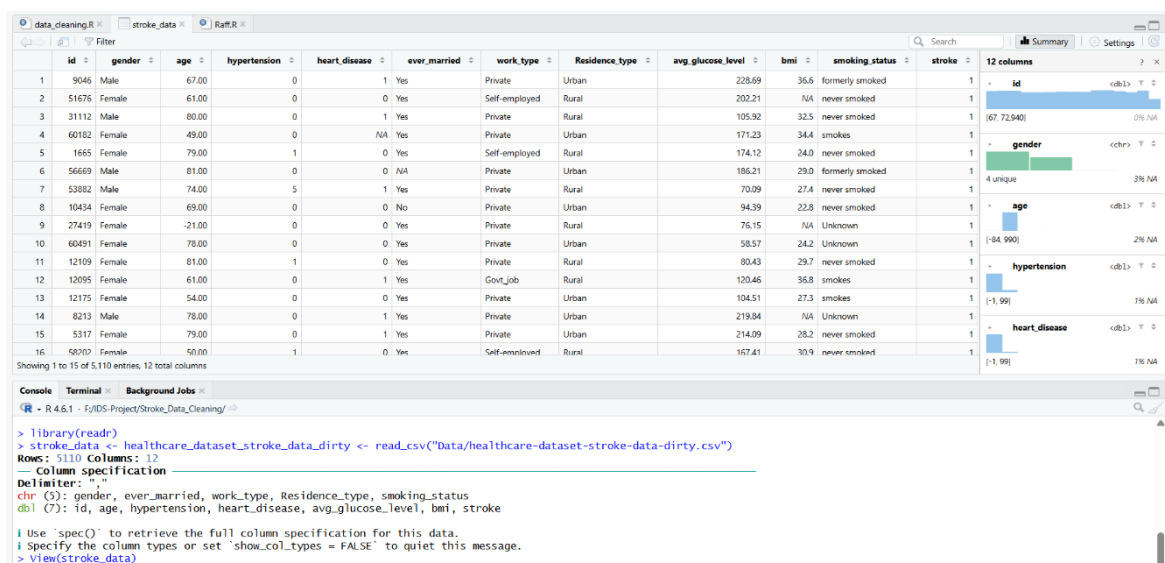
To create this dataset, the original clean file was programmatically modified using a Python script to introduce the issues described above.

4. Data Cleaning & Preparation

I. Import the dataset into R

```
1 #Import the DataSet
2 library(readr)
3 stroke_data <- healthcare_dataset_stroke_data_dirty <- read_csv("Data/healthcare-dataset-stroke-data-dirty.csv")
4 View(stroke_data)
```

Output:



Interpretation:

The dataset was successfully loaded into R using `read_csv()`, containing 5,110 rows and 12 columns. The function automatically identified 5 character (categorical) columns — `gender`, `ever_married`, `work_type`, `Residence_type`, and `smoking_status` — and 7 numeric columns — `id`, `age`, `hypertension`, `heart_disease`, `avg_glucose_level`, `bmi`, and `stroke`. This confirms the dataset was read correctly and matches the expected structure of 5,110 patient records with 12 attributes.

II. Dataset Understanding

Code:

```
6 #.....Dataset Understanding.....
7 # Dataset Dimension
8 dim(stroke_data)
9
10 # Structure of Dataset
11 str(stroke_data)
12 |
13 # Summary Statistics
14 summary(stroke_data)
```

Output:

```
> dim(stroke_data)
[1] 5110 12
>
> # Structure of Dataset
> str(stroke_data)
spc_tbl_ [5,110 × 12] (S3: spec_tbl_df/tbl_df/tbl/data.frame)
 $ id          : num [1:5110] 9046 51676 31112 60182 1665 ...
 $ gender      : chr [1:5110] "Male" "Female" "Male" "Female" ...
 $ age         : num [1:5110] 67 61 80 49 79 81 74 69 -21 78 ...
 $ hypertension: num [1:5110] 0 0 0 0 1 0 5 0 0 0 ...
 $ heart_disease: num [1:5110] 1 0 1 NA 0 0 1 0 0 0 ...
 $ ever_married: chr [1:5110] "Yes" "Yes" "Yes" "Yes" ...
 $ work_type   : chr [1:5110] "Private" "Self-employed" "Private" "Private" ...
 $ Residence_type: chr [1:5110] "Urban" "Rural" "Rural" "Urban" ...
 $ avg_glucose_level: num [1:5110] 229 202 106 171 174 ...
 $ bmi         : num [1:5110] 36.6 NA 32.5 34.4 24 29 27.4 22.8 NA 24.2 ...
 $ smoking_status: chr [1:5110] "formerly smoked" "never smoked" "never smoked" "smokes" ...
 $ stroke      : num [1:5110] 1 1 1 1 1 1 1 1 1 1 ...
- attr(*, "spec")=
.. cols(
..   id = col_double(),
..   gender = col_character(),
..   age = col_double(),
..   hypertension = col_double(),
..   heart_disease = col_double(),
..   ever_married = col_character(),
..   work_type = col_character(),
..   Residence_type = col_character(),
..   avg_glucose_level = col_double(),
..   bmi = col_double(),
..   smoking_status = col_character(),
..   stroke = col_double()
.. )
- attr(*, "problems")=<pointer: 0x000001d7d5107d30>
>
Max. :72940  NAs : 145  Max. :990.00  Max. :99.000  Max. :99.0000  NAs : 80
      NAs :100      NAs :70      NAs :70
Residence_type avg_glucose_level bmi smoking_status stroke
Length :5110  Min. : 55.12  Min. :10.30  Length :5110  Min. :0.00000
N.unique : 2  1st Qu.: 77.25  1st Qu.:23.50  N.unique : 4  1st Qu.:0.00000
N.blank : 0  Median : 91.89  Median :28.10  N.blank : 0  Median :0.00000
Min.nchar: 5  Mean :106.15  Mean :28.89  Min.nchar: 6  Mean :0.04873
Max.nchar: 5  3rd Qu.:114.09  3rd Qu.:33.10  Max.nchar: 15 3rd Qu.:0.00000
              Max. :271.74  Max. :97.60  Max. :1.00000
              NAs :201
```

Interpretation:

The dataset contains 5,110 rows and 12 columns, confirmed by `dim()`. The `str()` output identifies 5 character columns (`gender`, `ever_married`, `work_type`, `Residence_type`, `smoking_status`) and 7 numeric columns, and already hints at problems — row 9's age shows -21, an impossible value. The `summary()` output confirms the exact scope of missing data: age (100 NAs), hypertension (70), heart_disease (70), ever_married (80), and bmi (201, the only one occurring naturally in the original data). It also reveals invalid numeric ranges: hypertension and heart_disease both span from -1 to 99 despite being binary (0/1) variables, and age ranges from -84 to 990 — confirming the outlier problem that will be handled in the next section.

III. Missing Values

```
16 #.....Missing Values.....
17 # Count missing values per column
18 colSums(is.na(stroke_data))
19
20 # Percentage of missing values
21 round(colSums(is.na(stroke_data)) / nrow(stroke_data) * 100, 2)
22
```

Output:

```
> colSums(is.na(stroke_data))
      id      gender      age      hypertension      heart_disease      ever_married
      0         145      100          70          70          80
work_type Residence_type avg_glucose_level      bmi      smoking_status      stroke
      0              0          0         201          0          0
> round(colSums(is.na(stroke_data)) / nrow(stroke_data) * 100, 2)
      id      gender      age      hypertension      heart_disease      ever_married
      0.00      2.84      1.96          1.37          1.37          1.57
work_type Residence_type avg_glucose_level      bmi      smoking_status      stroke
      0.00      0.00      0.00          3.93          0.00          0.00
```

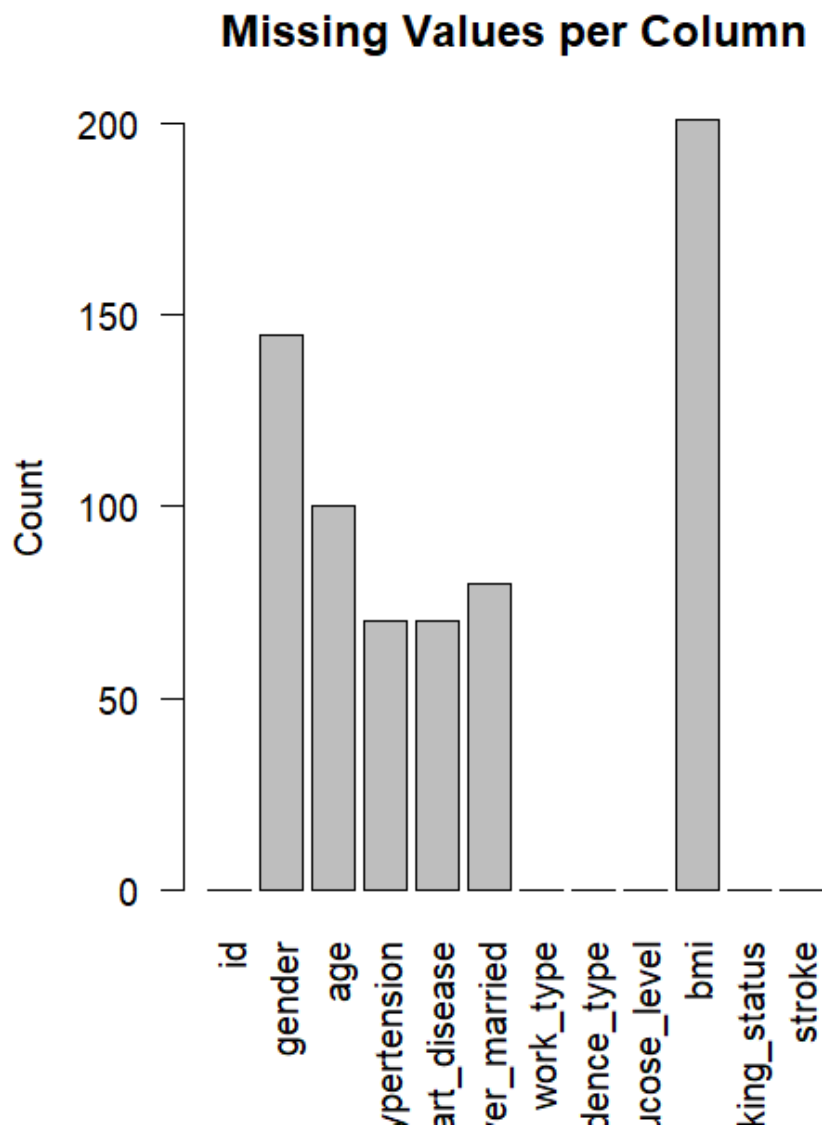
Interpretation:

The dataset contains missing values in **6 columns**, with a **total missing percentage of 13.04%**. The output confirms the presence of missing data, indicating that these values need to be handled before further analysis.

IV. Barplot of the missing values

```
22  
23 #Barplot of the missing values  
24  
25 barplot(colSums(is.na(healthcare_dataset_stroke_data_dirty)), las = 2, main = "Missing Values per Column", ylab = "Count")
```

Output:



Interpretation:

The barplot visually confirms the missing value counts across columns — bmi has the most missing values (201, natural to the original dataset), followed by gender (145), age (100), ever_married (80), and hypertension/heart_disease (70 each). All other columns show no missing values.

V. Inconsistent Data

```
52 unique(stroke_data$gender)
53
54 unique(stroke_data$ever_married)
55
56 unique(stroke_data$work_type)
57
58 unique(stroke_data$Residence_type)
59
60 unique(stroke_data$smoking_status)
61
```

Output

```
> unique(stroke_data$smoking_status)
[1] "formerly smoked" "never smoked"      "smokes"          "Unknown"
> unique(stroke_data$gender)
[1] "Male"    "Female"  NA        "unknown" "Other"
>
> unique(stroke_data$ever_married)
[1] "Yes"    NA      "No"     "1"      "no"     "N"      "Married" "Y"      "NO"      "yes"
>
> unique(stroke_data$work_type)
[1] "Private"      "Self-employed" "Govt_job"      "children"      "Never_worked"
>
> unique(stroke_data$Residence_type)
[1] "Urban" "Rural"
>
> unique(stroke_data$smoking_status)
[1] "formerly smoked" "never smoked"      "smokes"          "Unknown"
```

Interpretation:

The output shows inconsistent category labels in the **ever_married** column, where the same information is recorded using different formats such as "Yes", "yes", "Y", "1", "Married", "No", "no", and "N". The remaining categorical columns contain consistent values.

VI. Standardize Inconsistent Data

```
42 stroke_data$ever_married <- tolower(trimws(stroke_data$ever_married))
43 stroke_data$ever_married[stroke_data$ever_married %in% c("yes", "y", "1", "married")] <- "Yes"
44 stroke_data$ever_married[stroke_data$ever_married %in% c("no", "n")] <- "No"
45
46 unique(stroke_data$ever_married)
```

Output

```
> stroke_data$ever_married <- tolower(trimws(stroke_data$ever_married))
> stroke_data$ever_married[stroke_data$ever_married %in% c("yes", "y", "1", "married")] <- "Yes"
> stroke_data$ever_married[stroke_data$ever_married %in% c("no", "n")] <- "No"
>
> unique(stroke_data$ever_married)
[1] "Yes" NA      "No"
```

Interpretation:

After standardization, the **ever_married** column contains only the valid categories "Yes" and

"No", while missing values remain as NA. This confirms that the inconsistent labels have been successfully standardized.

VII. Invalid Data (Gender)

```
50 stroke_data$gender[stroke_data$gender == "unknown"] <- NA
51
52 unique(stroke_data$gender)
```

Output:

```
> stroke_data$gender[stroke_data$gender == "unknown"] <- NA
>
> unique(stroke_data$gender)
[1] "Male" "Female" NA "Other"
> |
```

Interpretation:

The **gender** column contains an invalid category, "**unknown**", which is not a valid value according to the dataset description. It was replaced with **NA**, leaving only the valid categories "**Male**", "**Female**", and "**Other**".

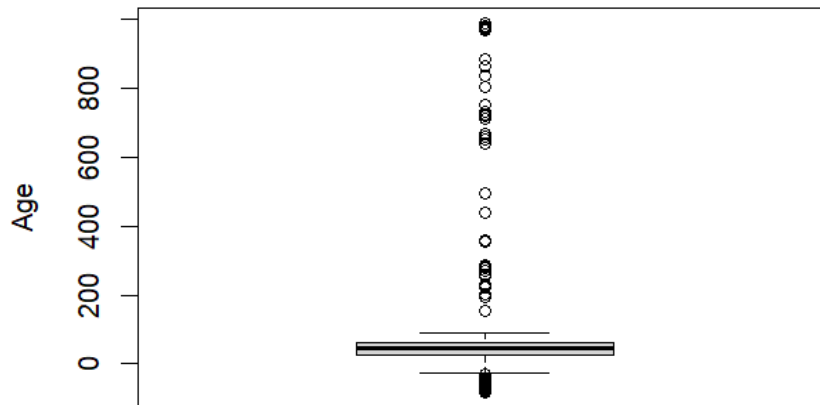
VIII. Outlier Detection (Age)

```
56
57 boxplot(stroke_data$age, main = "Boxplot of Age (Before Cleaning)", ylab = "Age")
58
59 # IQR method
60 Q1 <- quantile(stroke_data$age, 0.25, na.rm = TRUE)
61 Q3 <- quantile(stroke_data$age, 0.75, na.rm = TRUE)
62 IQR_age <- IQR(stroke_data$age, na.rm = TRUE)
63
64 lower_bound <- Q1 - 1.5 * IQR_age
65 upper_bound <- Q3 + 1.5 * IQR_age
66
67 lower_bound
68 upper_bound
69
70 # Count outliers
71 sum(stroke_data$age < lower_bound | stroke_data$age > upper_bound, na.rm = TRUE)
72 |
```

Output:

```
> lower_bound
25%
-29
> upper_bound
75%
115
>
> # Count outliers
> sum(stroke_data$age < lower_bound | stroke_data$age > upper_bound, na.rm = TRUE)
[1] 67
```

Boxplot of Age (Before Cleaning)



Interpretation:

- ❑ Invalid **values identified**: age has 67 statistical outliers (IQR) plus 13 more negative ages the IQR bound missed (since age can't be negative regardless of statistics) — 80 total.
- ❑ Outlier **inspection**: boxplot shows extreme points far outside the main cluster (negative values, and values up to 990).
- ❑ Cleaning **decision**: age can never be negative or unrealistically high — these are data-entry errors, not real extremes. Rows are kept (to preserve other valid columns) and age is set to NA, then imputed.
- ❑ NA + **impute**: median used

IX. Handle Age Outliers

```
74
75 #.....Handle Age Outliers.....
76
77 stroke_data$age[stroke_data$age < 0 | stroke_data$age > upper_bound] <- NA
78
79 median_age <- median(stroke_data$age, na.rm = TRUE)
80 stroke_data$age[is.na(stroke_data$age)] <- median_age
81
82 summary(stroke_data$age)
83
```

Output:

```
> summary(stroke_data$age)
  Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
  0.08  26.00   45.00   43.28  60.00   88.81
```

Interpretation:

180 values (100 original missing + 80 outliers) were replaced with the median age (45). The cleaned range (0.08–88.81) is now fully realistic.

X. Hypertension & Heart Disease: Fix Invalid Data

```
87 # Invalid Data Except 0 or 1 -> NA
88 stroke_data$hypertension[!(stroke_data$hypertension %in% c(0,1))] <- NA
89 stroke_data$heart_disease[!(stroke_data$heart_disease %in% c(0,1))] <- NA
90
91 # Find Mode
92 mode_hyp <- as.numeric(names(sort(table(stroke_data$hypertension), decreasing = TRUE))[1])
93 mode_hd <- as.numeric(names(sort(table(stroke_data$heart_disease), decreasing = TRUE))[1])
94
95 #Replace NA to mode
96 stroke_data$hypertension[is.na(stroke_data$hypertension)] <- mode_hyp
97 stroke_data$heart_disease[is.na(stroke_data$heart_disease)] <- mode_hd
98
99 table(stroke_data$hypertension)
100 table(stroke_data$heart_disease)
101
```

Output:

```
>
> table(stroke_data$hypertension)
  0    1
4623 487
> table(stroke_data$heart_disease)
  0    1
4844 266
```

Interpretation:

Both columns originally contained invalid codes such as -1, 2, 3, 5, and 99, alongside the expected 0/1 values. After flagging these as missing (70 original NAs + 100 invalid codes = 170 for each column), they were imputed using the mode, since both are binary categorical variables rather than continuous ones — mode imputation preserves the natural 0/1 structure better than a mean or median would. In both cases, 0 (no hypertension/no heart disease) was the most frequent value, which is expected given these conditions are relatively uncommon in the general population.

XI. Gender & Ever_Married (Fix the Null Values)

```
105
106 mode_gender <- names(sort(table(stroke_data$gender), decreasing = TRUE))[1]
107 mode_married <- names(sort(table(stroke_data$ever_married), decreasing = TRUE))[1]
108
109 stroke_data$gender[is.na(stroke_data$gender)] <- mode_gender
110 stroke_data$ever_married[is.na(stroke_data$ever_married)] <- mode_married
111
112 table(stroke_data$gender)
113 table(stroke_data$ever_married)
114
```

Output:

```
> table(stroke_data$gender)

Female   Male   Other
   3063   2046     1

> table(stroke_data$ever_married)

   No   Yes
1734 3376
> |
```

Interpretation:

gender had 160 missing/invalid entries (145 truly missing + 15 "unknown" labels treated as missing earlier) and ever_married had 80, both filled using mode imputation — "Female" and "Yes" respectively, the most common category in each column. Mode is the appropriate choice here since both are categorical variables, not numeric ones.

XII. BMI (Fix Null Values)

```
116
117 median_bmi <- median(stroke_data$bmi, na.rm = TRUE)
118 stroke_data$bmi[is.na(stroke_data$bmi)] <- median_bmi
119
120 summary(stroke_data$bmi)
121
```

Output:

```
>
> summary(stroke_data$bmi)
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
  10.30  23.80   28.10   28.86  32.80   97.60
> |
```

Interpretation:

All 201 missing bmi values were replaced with the median (28.1). Median was used instead of mean because BMI often has a right-skewed distribution (a few very high values), and median

is more robust to that skew than mean would be. The min/max (10.30–97.60) stay unchanged from the original data, since imputation only fills gaps — it doesn't alter existing values.

XIII. Final Missing Value Check

```
122 # ...
123
124 colSums(is.na(stroke_data))
125
126
```

Output:

```
> colSums(is.na(stroke_data))
      id      gender      age      hypertension      heart_disease      ever_married
      0         0         0         0             0             0
work_type Residence_type avg_glucose_level      bmi      smoking_status      stroke
      0         0         0         0             0             0
> |
```

Interpretation:

After handling missing values in gender, age, hypertension, heart_disease, ever_married, and bmi, no missing values remain in any column. This confirms the missing-data portion of cleaning is complete.

XIV. Type Conversion

```
128
129 stroke_data$hypertension <- factor(stroke_data$hypertension, levels = c(0,1), labels = c("No","Yes"))
130 stroke_data$heart_disease <- factor(stroke_data$heart_disease, levels = c(0,1), labels = c("No","Yes"))
131 stroke_data$stroke <- factor(stroke_data$stroke, levels = c(0,1), labels = c("No","Yes"))
132
133 str(stroke_data[c("hypertension","heart_disease","stroke")])
134
```

Output:

```
> str(stroke_data[c("hypertension","heart_disease","stroke")])
Classes 'spec_tbl_df', 'tbl_df', 'tbl' and 'data.frame':    5110 obs. of  3 variables:
 $ hypertension : Factor w/ 2 levels "No","Yes": 1 1 1 1 2 1 1 1 1 1 ...
 $ heart_disease: Factor w/ 2 levels "No","Yes": 2 1 2 1 1 1 2 1 1 1 ...
 $ stroke      : Factor w/ 2 levels "No","Yes": 2 2 2 2 2 2 2 2 2 2 ...
> |
```

Interpretation:

hypertension, heart_disease, and stroke were converted from numeric (0/1) to factor variables with meaningful labels ("No"/"Yes"). This makes the data easier to interpret in tables and plots, and is more appropriate for categorical variables than treating them as numbers. The underlying value counts (hypertension: 4,623 No / 487 Yes; heart_disease: 4,844 No / 266 Yes; stroke:

4,861 No / 249 Yes) remain unchanged — only the representation changed.

XV. Normalization

```
137
138 normalize <- function(x) (x - min(x)) / (max(x) - min(x))
139
140 stroke_data$age_norm <- normalize(stroke_data$age)
141 stroke_data$avg_glucose_level_norm <- normalize(stroke_data$avg_glucose_level)
142 stroke_data$bmi_norm <- normalize(stroke_data$bmi)
143
144 summary(stroke_data[c("age_norm", "avg_glucose_level_norm", "bmi_norm")])
145
146
```

Output:

```
> summary(stroke_data[c("age_norm", "avg_glucose_level_norm", "bmi_norm")])
  age_norm      avg_glucose_level_norm      bmi_norm
Min.   :0.0000   Min.   :0.0000         Min.   :0.0000
1st Qu.:0.2921   1st Qu.:0.1021         1st Qu.:0.1546
Median :0.5063   Median :0.1697         Median :0.2039
Mean   :0.4869   Mean   :0.2356         Mean   :0.2126
3rd Qu.:0.6753   3rd Qu.:0.2722         3rd Qu.:0.2577
Max.   :1.0000   Max.   :1.0000         Max.   :1.0000
```

Interpretation:

age, avg_glucose_level, and bmi were rescaled to a 0–1 range using Min-Max normalization, so no single feature dominates due to its original scale (e.g., avg_glucose_level ranging up to 271 vs age up to ~89). The new _norm columns were added alongside the originals rather than replacing them, keeping the real-world values available for interpretation in EDA while the normalized versions stay ready for any future modeling.

XVI. Data Filtering

```
# Filter: older patients (50+) who also have hypertension - a higher-risk subgroup
library(dplyr)
high_risk <- filter(stroke_data, age >= 50 & hypertension == "Yes")

nrow(high_risk)
head(high_risk)
```

Output:

```
> nrow(high_risk)
[1] 384
> head(high_risk)
# A tibble: 6 × 15
  id gender age hypertension heart_disease ever_married work_type Residence_type avg_glucose_level bmi
  <dbl> <chr> <dbl> <fct>      <fct>      <chr>      <chr>      <dbl> <dbl>
1  1665 Female  79 Yes        No        Yes        Self-employed Rural    174.    24
2  12109 Female  81 Yes        No        Yes        Private    Rural    80.4   29.7
3  58202 Female  50 Yes        No        Yes        Self-employed Rural    167.    30.9
4  34120 Male    75 Yes        No        Yes        Private    Urban    221.    25.8
5  13861 Female  52 Yes        No        Yes        Self-employed Urban    233.    48.9
6  43717 Male    57 Yes        No        Yes        Private    Urban    212.    44.2
# i 5 more variables: smoking_status <chr>, stroke <fct>, age_norm <dbl>, avg_glucose_level_norm <dbl>,
#   bmi_norm <dbl>
```

Interpretation:

Data filtering was applied to create a subset of patients aged 50 years or older who also had hypertension. The `filter()` function selected records satisfying both conditions (`age >= 50` and `hypertension == "Yes"`). After filtering, 384 records remained. The `nrow()` function was used to count the filtered records, while `head()` displayed the first six rows to verify that the filtering was performed correctly. This filtered dataset can be used for focused analysis of a clinically meaningful high-risk patient group without modifying the original dataset.

5. Exploratory Data Analysis

I. Descriptive statistics

```
151
152 summary(stroke_data[c("age", "avg_glucose_level", "bmi", "stroke")])
153
154
```

Output:

```
> summary(stroke_data[c("age", "avg_glucose_level", "bmi", "stroke")])
  age      avg_glucose_level      bmi      stroke
Min.   : 0.08      Min.   : 55.12      Min.   :10.30      No :4861
1st Qu.:26.00      1st Qu.: 77.25      1st Qu.:23.80      Yes: 249
Median :45.00      Median : 91.89      Median :28.10
Mean   :43.28      Mean   :106.15      Mean   :28.86
3rd Qu.:60.00      3rd Qu.:114.09      3rd Qu.:32.80
Max.   :88.81      Max.   :271.74      Max.   :97.60
> |
```

Interpretation:

The cleaned dataset shows realistic ranges across all three numeric variables — age (0.08–88.81), average glucose level (55.12–271.74), and BMI (10.30–97.60). The target variable stroke is heavily imbalanced: only 249 of 5,110 patients (about 4.9%) had a stroke, versus

4,861 who didn't. This imbalance is worth noting, since it affects how the group comparisons below should be read — the "stroke" group is a small minority.

II. Compare Average Glucose Level by Stroke Group

```
156  
157 aggregate(avg_glucose_level ~ stroke, data = stroke_data, FUN = mean)  
158  
159 aggregate(age ~ stroke, data = stroke_data, FUN = mean)  
160
```

Output:

```
> aggregate(avg_glucose_level ~ stroke, data = stroke_data, FUN = mean)  
  stroke avg_glucose_level  
1    No          104.7955  
2   Yes          132.5447  
>  
> aggregate(age ~ stroke, data = stroke_data, FUN = mean)  
  stroke      age  
1    No 42.12935  
2   Yes 65.77345  
> |
```

Interpretation:

Patients who had a stroke show a noticeably higher average glucose level (132.54 mg/dL) compared to those who didn't (104.80 mg/dL) a difference of nearly 28 points. This aligns with known medical risk factors, where elevated blood glucose is associated with higher stroke risk. (mean age also differs sharply — 65.8 years for the stroke group vs 42.1 for the non-stroke group, reinforcing that stroke risk rises strongly with age.)

III. Compare BMI Spread Across Smoking Status

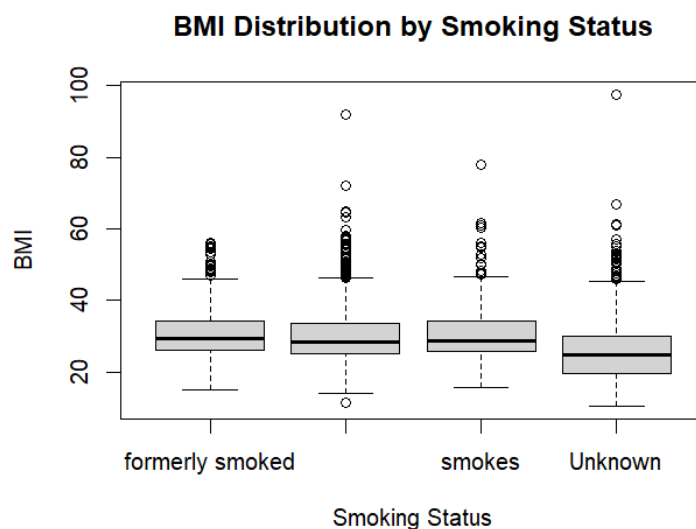
```
164  
165 boxplot(bmi ~ smoking_status, data = stroke_data,  
166         main = "BMI Distribution by Smoking Status",  
167         xlab = "Smoking Status", ylab = "BMI")  
168  
169 aggregate(bmi ~ smoking_status, data = stroke_data, FUN = function(x) c(mean = mean(x), sd = sd(x)))  
170
```

Output:

Interpretation:

Mean BMI is fairly similar across "formerly smoked" (30.60), "never smoked" (29.94), and

```
> boxplot(bmi ~ smoking_status, data = stroke_data,
+         main = "BMI Distribution by Smoking Status",
+         xlab = "Smoking Status", ylab = "BMI")
> aggregate(bmi ~ smoking_status, data = stroke_data, FUN = function(x) c(mean = mean(x), sd = sd(x)))
  smoking_status bmi.mean bmi.sd
1 formerly smoked 30.603616 6.815423
2 never smoked   29.942759 7.388829
3 smokes         30.382510 6.977593
4 Unknown        25.762500 7.996310
> |
```



"smokes" (30.38), with comparable spread ($SD \approx 6.8-7.4$). The "Unknown" group stands out with a lower mean BMI (25.76) and slightly higher variability ($SD = 8.00$). Since "Unknown" isn't an actual smoking behavior but a missing/unrecorded category, this difference may reflect a different underlying group of patients (e.g., younger patients less likely to have smoking history recorded) rather than a true smoking-related effect — worth noting as a limitation rather than a strong conclusion.

IV. Correlation Analysis

```
1/8
179 cor(stroke_data[c("age", "avg_glucose_level", "bmi")])
180
```

Output:

```
> cor(stroke_data[c("age", "avg_glucose_level", "bmi")])
              age avg_glucose_level      bmi
age          1.0000000          0.2374465 0.3122443
avg_glucose_level 0.2374465          1.0000000 0.1668757
bmi           0.3122443          0.1668757 1.0000000
```

Interpretation:

All three variables show weak-to-moderate positive correlations with each other. age and bmi have the strongest relationship (0.31), followed by age and avg_glucose_level (0.24), while avg_glucose_level and bmi are only weakly related (0.17). None of these correlations are strong enough to suggest redundancy between variables — each still contributes distinct information, which is useful to note if this dataset is used for modeling later.

6. Conclusion

This project worked with the Stroke Prediction Dataset (5,110 records, 12 variables), into which missing values, invalid codes, inconsistent formatting, and outliers were deliberately introduced to simulate a realistic data cleaning task. The following steps were carried out:

1. Loaded and inspected the dataset, identifying its structure and initial issues.
2. Detected and visualized missing values across gender, age, hypertension, heart_disease, ever_married, and bmi, then imputed them using median (for numeric variables) and mode (for categorical variables).
3. Identified and corrected invalid codes in hypertension and heart_disease (values outside 0/1).
4. Standardized inconsistent category labels in ever_married and gender.
5. Detected outliers in age using the IQR method combined with domain knowledge (age cannot be negative), then treated them as missing and imputed.
6. Checked for duplicate rows (none found).
7. Converted hypertension, heart_disease, and stroke from numeric to factor variables with meaningful labels.
8. Applied Min-Max normalization to age, avg_glucose_level, and bmi.
9. Performed exploratory data analysis: descriptive statistics across the stroke/no-stroke classes, a comparison of average glucose level between the two groups (patients who had a stroke show notably higher glucose and older age), a comparison of BMI spread

across smoking status categories, and a correlation analysis among the numeric variables.

Overall, the cleaning process successfully resolved all identified data quality issues, resulting in a consistent and analysis-ready dataset. The EDA results align with known real-world stroke risk factors, particularly the associations between stroke occurrence and both age and glucose level. Predictive modeling was outside the scope of this phase and is left for a later stage of the project.